NODEBASE — INTERNAL ENGINEERING DOCUMENT

The `account` Table

What `providerId` and `accountId` actually hold, why the token columns are nullable, what `scope` means, whether email-and-password sign-in touches any of it, and who — if anyone — reads those tokens afterwards.

Table — `account` (`@@map` , line 65) **Rows measured** — 19, across 3 providers

Evidence — live Neon reads + installed `better-auth` source

Relationship to the other documents. The `account` table is the join between a `user` and a way of proving who they are. It is a *different* table from the `Credential` table covered in `credential-save-flow.pdf` , which stores third-party API keys the user supplies for workflow nodes. They are frequently confused because both concern "secrets" and both relate to a user — but they are unrelated, have different schemas, different lifecycles, and different security properties. Sign-up and OTP mechanics are in `otp-email-flow.pdf` .

1. Short answers

QUESTION	ANSWER
What is `providerId` ?	A free-text **discriminator naming which authentication method produced this row**. It is `text` in Postgres — not an enum, no check constraint, no index. Values in this database: `credential` , `google` , `github` .
What is `accountId` ?	**The identity that provider issued**, and the column that makes a provider link unique. For `credential` it is a copy of `userId` (14/14 rows). For OAuth it is the provider's own numeric user id as a string — Google's `115737422360838922756` , GitHub's `150208691` .
Why are the token columns optional?	Because a provider only issues tokens if you asked for API access, and this project never does. **All 19 rows have `refreshToken = null`** , including all 4 Google and 1 GitHub logins. A `credential` row has no concept of a token.
What is `scope` ?	The permissions the provider *granted*, comma-joined, and the record of what was actually consented to rather than requested. Written as `scope: tokens.scopes?.join(",")` .
Does email+password use access/refresh tokens?	**No.** All 14 `credential` rows have `accessToken` , `refreshToken` , `idToken` , `scope` and both expiry columns all `null` . A session cookie is the only token involved.
Who uses the tokens?	**Nobody in this application.** No application code queries the `account` table at all — zero references to `prisma.account` . They are written at login and never read back.

2. The table as it exists

Thirteen columns, defined at `prisma/schema.prisma:49-66` . The Postgres types below were read from `information_schema` , not inferred from Prisma:

COLUMN	PRISMA	POSTGRES	NULL?
id	String @id	text	NOT NULL (primary key)
accountId	String	text	**NOT NULL**
providerId	String	text	**NOT NULL**
userId	String	text	**NOT NULL** (FK)
accessToken	String?	text	nullable
refreshToken	String?	text	nullable
idToken	String?	text	nullable
accessTokenExpiresAt	DateTime?	timestamp	nullable
refreshTokenExpiresAt	DateTime?	timestamp	nullable
scope	String?	text	nullable
password	String?	text	nullable
createdAt	DateTime	timestamp	NOT NULL
updatedAt	DateTime	timestamp	NOT NULL

Only **two** constraints exist, read from `pg_constraint`:

```
account_pkey       PRIMARY KEY (id)
account_userId_fkey FOREIGN KEY ("userId") REFERENCES "user"(id)
                   ON UPDATE CASCADE ON DELETE CASCADE
```

There is no unique index on `(providerId, accountId)`

That pair is the logical identity of a provider link — the thing that answers "has this person already linked their Google account?" — and the library itself uses it that way when linking. But the database does not enforce it, and nothing indexes it. Consequences: the same Google `sub` can be inserted twice for one user, and every lookup that should hit an index instead does a sequential scan. Adding `@@unique([providerId, accountId])` costs one index and closes both. (Measured via `pg_constraint`; the index is genuinely absent rather than merely unnamed.)

The `onDelete: Cascade` on `userId` means deleting a user removes every account row automatically. Combined with the `Credential.userId` cascade, a user deletion takes their third-party API keys and their provider links with it, with no application code.

3. `providerId` — what it is and what can live in it

`providerId` answers one question: **which authentication method created this row?** It is a discriminator in a table that models a one-to-many relationship, and it is deliberately unconstrained.

What is in this database

LIVE — 19 ROWS GROUPED BY PROVIDER

```
=== account, GROUP BY "providerId" ===

providerId     rows  pwd  access  refresh  idTok  scope  aExp  rExp  acctId=userId
credential      14   14       0        0      0      0     0     0        14/14
google           4    0       4        0      4      4     4     0         0/4
github           1    0       1        0      0      1     0     0         0/1
              â”€â”€â”€
              19   14       5        0      4      5     4     0
```

The three-way split is the whole design in one table. `credential` rows are password rows; OAuth rows are token rows; and the two never mix — **not one OAuth row has a password, and not one credential row has a token.**

What else could live in it

Because the column is unconstrained text, the value is whatever the code that inserts the row chose. Verified in the installed package:

SOURCE	VALUE WRITTEN	HOW IT IS CHOSEN
Email + password	`credential`	Hard-coded. `providerId: "credential"` with `accountId: user.user.id` (email-otp plugin, and the credential path at `api-CkmycQ2x.mjs:799`).
`socialProviders`	`google, github`	The **key** you write under `socialProviders` in `auth.ts:63-72` becomes the id. Rename the key to `work-google` and the rows become `work-google`.
`genericOAuth`	`auth0, hubspot, keycloak, microsoft-entra-id, okta, slack, line-jp, line-th, line-tw`	Each preset supplies its own `providerId`. Arbitrary strings work too — the plugin reads whatever you configure.
SIWE wallet plugin	`siwe`	Hard-coded `providerId: "siwe"`.
Passkey / other plugins	`plugin-defined`	Each plugin that creates an account row picks its own discriminator. Admin-style flows use `accountId: user.id` the way `credential` does.

> **The practical consequence of an unconstrained discriminator**
> There is no foreign key, no enum, and no check, so the database cannot tell you that a typo like `goolge` created a set of rows that no longer correspond to any configured provider. Orphaned `providerId` values are invisible to SQL and only discoverable by comparing the column against `auth.ts`. If you want the database to help, an enum or a check constraint against your known provider list is the cheapest option — though note it makes adding a provider a migration.

4. `accountId` — the provider's own identifier

`accountId` is the identifier *the provider issued*, stored as text. It is the value that lets a returning visitor be recognised on their second login, and it is deliberately allowed to differ from `userId` — that difference is the entire reason the `account` table exists rather than a password column on `user`.

Two completely different shapes

From the OAuth callback (`api-CkmycQ2x.mjs:1075`):

```
const accountData = {
  providerId: provider.id,
  accountId: String(userInfo.id),   // â†⍰ the provider's own user id
  ...tokens,
  scope: tokens.scopes?.join(",")
};
```

And for the credential path (`api-CkmycQ2x.mjs:799`) it is simply the user's own id copied across. Both are visible in the live data:

LIVE — ONE REPRESENTATIVE ROW PER PROVIDER

```
=== accountId SHAPE PER PROVIDER ===

credential
  accountId : jRAZttIaGUe12DcYMteksPQyLHKIc8j3
  userId    : jRAZttIaGUe12DcYMteksPQyLHKIc8j3
  same?     : true   // 14/14 rows â€” a self-link

google
  accountId : 115737422360838922756        // Google's "sub" claim
  userId    : Do3NC8BrCovHjXkm3e8vsNryuQTYwUpy
  same?     : false  // 0/4 rows

github
  accountId : 150208691                   // GitHub's numeric user id
  userId    : W8Py91QRQ50Vdhy6MGikI2sbhG0RWJXP
  same?     : false  // 0/1 rows
```

PROVIDER	WHAT THE NUMBER IS	PROPERTIES AND CAUTIONS
`credential`	Your own internal user id, duplicated	Stable, opaque, and app-generated. The duplication is not redundant — it is what lets the `credential` row sit in the same table and participate in the same linking logic. The library is explicit that it is a copy, not a separate identity.
`google`	The `sub` claim from the OIDC ID token — a 21-22 digit string	Google's `sub` is per-*client*: the same human gets a **different** `sub` in a different OAuth client. So the value is only meaningful together with `providerId` — which is exactly why they are a composite identity, and why the missing unique index on the pair matters.
`github`	GitHub's numeric account id	Also *not* the GitHub username — usernames can be changed and reassigned; the numeric id cannot. Storing the id rather than the handle is the correct choice, and it is why the column is `text` rather than a username type.

Why keeping them separate is not over-engineering

Because `accountId` is provider-owned, one `user` row can carry several of them. Measured — **3 users hold more than one account row**:

```
=== users with >1 account row ===
CpopolS6nWhtw4n5Le8gfzytpMzCVrqF -> [credential, google]
Do3NC8BrCovHjXkm3e8vsNryuQTYwUpy -> [credential, google]
W8Py91QRQ50Vdhy6MGikI2sbhG0RWJXP -> [github, google]
```

One of them signed up with a password and later added Google. A schema that put an identity in a column on `user` could not represent that at all. The cost is an extra indexed read per authentication; the benefit is that adding or removing a login method is a single-row insert or delete that never touches the user record.

5. Why the token columns are optional

All five token-related columns are `String?` / `DateTime?` in Prisma and `nullable` in Postgres. There are three independent reasons, and they stack:

1. **A credential row has no tokens to store**

   There is no third party in the loop. The user typed a password, Better Auth hashed it with scrypt, and the result went into `password`. No authorisation was delegated, so there is nothing to delegate back. Confirmed: **all 14 credential rows have every token column null**, including both expiry timestamps.

2. **Tokens are only written if the provider returns one**

   The insert is unconditional in shape but conditional in content — `accountId: String(userInfo.id), ...tokens` — so absent values land as `null` rather than erroring. On the linking path the library is even more careful (`api-CkmycQ2x.mjs:1050`):

   ```
   }).filter(([_, value]) => value !== void 0);
   ```

   so re-authorising with Google without a refresh token will not blank a refresh token you already had. Omitting the key beats writing `null`.

3. **Refresh tokens require asking for offline access**

   This is the substantive reason, and it is a configuration fact. A provider issues a refresh token only when the client requests offline access — for Google, `access_type=offline` plus `prompt=consent`. `auth.ts:63-72` configures `github` and `google` with `clientId` and `clientSecret` and nothing else, so no offline prompt is sent.

   The result, measured: `refreshToken` **is null on 19 of 19 rows** — all 4 Google and the GitHub row included. `idToken` splits the other way: present on all 4 Google rows (OIDC) and null on the GitHub row, because GitHub is OAuth 2.0 without OpenID Connect.

Consequence: every stored access token is already dead

Google access tokens are short-lived, and each row records when it lapsed. Comparing against today's date, **all four are in the past and none can be renewed**, because there is no refresh token to renew them with:

ROW CREATED	ACCESSTOKENEXPIRESAT	STATUS
2026-04-05	2026-04-18	expired
2026-04-16	2026-04-18	expired
2026-09-23	2026-09-23	expired
2026-09-26	2026-09-26	expired

The GitHub row has no expiry at all — GitHub tokens do not expire by default, so that column is legitimately null rather than missing. Nothing in the app reads these, so nothing breaks today; it matters the moment you add a feature that calls a Google or GitHub API on the user's behalf. See finding 2.

6. The `scope` field

`scope` records **which permissions the user actually granted** — the provider's answer, not your request. It is written as a comma-joined string:

```
scope: tokens.scopes?.join(",")
```

LIVE — EVERY NON-NULL SCOPE IN THE DATABASE

```
=== SCOPE VALUES ===

google  (4 rows, all populated)
  https://www.googleapis.com/auth/userinfo.profile,
  openid,
  https://www.googleapis.com/auth/userinfo.email

  // note: order varies between rows â€” the provider's order, not yours
  // 1 row stored them as:
  //   userinfo.profile, openid, userinfo.email
  // 3 rows stored them as:
  //   userinfo.profile, userinfo.email, openid

github  (1 row)
  read:user,user:email

credential (14 rows)
  null
```

SCOPE	MEANING
`openid`	Asks for an ID token. The marker of the OpenID Connect layer, and the reason `idToken` exists on Google rows but not GitHub ones.
`userinfo.email`	Read the user's email address — required, because Better Auth uses it to find or create the `user` row.
`userinfo.profile`	Read name and profile picture. Optional; Better Auth falls back to the email as the display name.
`read:user`	GitHub's read access to the account itself. Paired with `user:email` to expose the address.

Two things worth knowing about this column. First, it is a **record of consent, not a request**: if you request five scopes and the user approves two, this holds two. Second, **order is not stable** — three rows here store `openid` second and one stores it third, all from the same login flow. Anything that compares scope strings must split on commas and compare as a set, or it will produce spurious mismatches.

The scopes present are exactly the minimum Better Auth needs to identify a user. This project requests no API scopes — no Drive, no Gmail, no repo access. That is the correct posture for a sign-in-only integration, and it is also why no access token is actually needed after login.

7. Does email-and-password sign-in use access or refresh tokens?

No, and it is worth being precise about why, because "token" is overloaded in this codebase.

CREDENTIAL SIGN-IN — THE COMPLETE DATA PATH

1. **User submits email + password to `/sign-in/email`**
2. **Better Auth looks up the `user` row by email, then the `account` row where `providerId = "credential"` and `userId` matches**
3. **Verify the scrypt hash in constant time**

   `verifyPassword({ hash, password })` splits `salt:hash`, re-derives with PBKDF2-style scrypt at `N: 16384, r: 16, p: 1, dkLen: 64`, and compares with `constantTimeEqual`. No network call, no third party.
4. **Create a `session` row and set a cookie**

   The cookie *is* the token in this flow — a random session token stored in the `session` table and sent as an httpOnly cookie. It has nothing to do with OAuth. No `account` column is written at sign-in; the `credential` row was created once at sign-up and is read-only thereafter.

The three token-like things in this codebase, disambiguated

1. The session cookie — issued on every successful sign-in, verified on every request, stored in the `session` table. Used by both password and OAuth sign-in.

2. The OTP code — a 6-digit number, hashed into `verification`, used once during email verification. Not related to OAuth at all.

3. The OAuth access/refresh token — issued by Google or GitHub, stored in `account`, and **unused** in this project. Most "the app has a refresh token" sightings in this repo are actually the Inngest realtime subscribe token, which is a fourth thing entirely. See section 8.

8. Who uses the tokens?

Searching the entire application for reads of these columns returns nothing:

```
$ grep -rn "prisma\.account" src/
  (no matches)

$ grep -rn "accessToken|refreshToken|idToken" src/
  polar.ts          â†’ process.env.POLAR_ACCESS_TOKEN      (a server env var)
  16 node components â†’ refreshToken: fetchâ€¦RealtimeToken
                       (type: Realtime.Subscribe.Token â€” Inngest realtime)
```

No application code queries the `account` table. Zero matches for `prisma.account`. The eighteen `refreshToken` hits are all a different concept: the Inngest realtime subscribe token declared as `refreshToken: () => Promise<Realtime.Subscribe.Token>` in `use-node-status.ts:10`, used to reconnect the live status channel. Not OAuth.

CONSUMER	READS THE TOKENS?
The OAuth provider (Google, GitHub)	No — it issued them. They would be validated by a resource server such as the Google Sheets API, which this project never calls.
Better Auth internals	Only during the OAuth callback, to fetch profile info, and it stores the result rather than keeping the token. It also exposes a `getAccessToken` helper — **unused here**.
Your application code	**No.** Never read, at any point, for any provider.
Nodebase's `Credential` feature	No — entirely separate table, entirely separate flow. API keys the user supplies for workflow nodes never touch `account`.

Unencrypted bearer tokens sitting in a live database

Better Auth can encrypt these columns, but only if you ask: `account.encryptOAuthTokens` is opt-in, and `src/lib/auth.ts` has no `account` block at all. So the tokens are stored **in plaintext** — visible in the live data as `ya29.a0ARGnu0Z1AeN7iIZpprhVoeznv…` and `gho_mu72hHrYDiWTNBNN2LH5Cs7t8B4T…`. Setting the flag makes every write go through `symmetricEncrypt` with the server secret, and Better Auth decrypts transparently on read.

The comparison is unflattering: **passwords are scrypt-hashed, user API keys are AES-256-GCM encrypted, and OAuth bearer tokens are stored raw.** Since nothing reads them, the cheapest correct fix is to stop requesting them — request only `openid email profile` and no API scopes, so no access token is issued at all. The tokens are also already expired (section 5), so their current practical value is low, but they remain valid credentials for whatever they were scoped to.

9. Full flow

Path A — email and password (writes 1 account row)

1. `POST /sign-up/email` → **validate** → **create** `user` **row (** `emailVerified: false` **)**

2. **Create the `credential` account row**

```
createAccount({
  userId: user.user.id,
  providerId: "credential",
  accountId: user.user.id,   // â†  a copy of userId
  password: passwordHash     // â†  scrypt, 161 chars
})
```

No token, no scope, no expiry — because there is nothing to store.

3. **Email an OTP; store its hash in `verification` under `email-verification-otp-<email>`**

4. **User submits the code → `emailVerified: true`, verification row deleted, session created**

5. **`POST /sign-in/email` → find user, verify hash in constant time, create session**

Path B — OAuth (writes 1 account row, 4-5 columns)

1. **Redirect to Google/GitHub with `clientId`, state, PKCE**

2. **Callback with `?code=`. Provider exchanges it for tokens — this is where a refresh token *would* appear, had offline access been requested**

3. **Fetch the profile with `provider.getUserInfo({ ...tokens })`**

   `userInfo.id` becomes `accountId`; `userInfo.email` locates or creates the `user` row.

4. **Persist**

```
{
  providerId: provider.id,              // "google" | "github"
  accountId: String(userInfo.id),       // "115737422360838922756"
  accessToken, refreshToken, idToken,   // â†  plaintext, unencrypted
  accessTokenExpiresAt, refreshTokenExpiresAt,
  scope: tokens.scopes?.join(",")       // "openid,userinfo.email,â€¦"
}
```

5. **Session created and cookie set — the same mechanism as Path A**

Path C — linking an additional provider

1. **Guard checks, in order (`api-CkmycQ2x.mjs:1035-1042`)**

   Is the provider in `accountLinking.trustedProviders`? Is the account's email verified? Does it match the email you signed in with, unless `allowDifferentEmails`? Does the provider account already belong to a *different* user?

2. **Update the existing row, or insert a new one**

   If the provider account is already linked to you, the tokens are refreshed in place and undefined values are filtered out. Otherwise a second `account` row is created for the same `userId` — the multi-provider case measured in section 4.

Note on the linking guard and this project

`auth.ts` sets no `accountLinking` configuration, so Better Auth's defaults apply. Worth knowing that automatic linking by matching verified email is the behaviour to review before adding a provider — a misconfigured or untrusted provider that returns a verified email could attach itself to an existing account. The library requires the provider to be in `trustedProviders` and the email to be verified, but the default set is worth confirming explicitly rather than inheriting.

10. Findings

#	FINDING	SEVERITY	DETAIL AND SUGGESTED FIX
1	OAuth tokens stored unencrypted	High	`encryptOAuthTokens` is opt-in and unset, so access and ID tokens are plaintext — while passwords are scrypt-hashed and user API keys are AES-256-GCM encrypted. A database leak yields usable Google/GitHub bearer tokens. Set `account: { encryptOAuthTokens: true }`, or better, request only `openid email profile` so no API token is issued at all.
2	Every stored access token is already expired and unrefreshable	Medium	All 4 Google `accessTokenExpiresAt` values are in the past; `refreshToken` is null on 19/19 rows because `access_type=offline` is not requested. Harmless today because nothing reads them, but any future "call Google on the user's behalf" feature will fail on first use. Decide deliberately whether you want offline access.
3	No unique index on `(providerId, accountId)`	Medium	The logical key of a provider link is unenforced and unindexed. Duplicate links are possible and lookups cannot use an index. Add `@@unique([providerId, accountId])`.
4	Tokens are retained after they are useless	Low	Dead tokens accumulate indefinitely — 4 stale Google tokens here, oldest expired over five months. If you keep sign-in-only, consider not requesting them; otherwise clear expired ones periodically.
5	`providerId` is unconstrained text	Low	No enum or check, so a typo or removed provider leaves orphan values the database cannot detect. Compare the column against `auth.ts` in a periodic check, or add a constraint.
6	`scope` order is not stable	Informational	Measured: 3 Google rows store `openid` second, 1 stores it third, from the same code path. Compare scopes as sets, never as strings.

11. Reference

LOCATION	ROLE
`prisma/schema.prisma:49-66`	The `Account` model. No `@@unique` — see finding 3.
`src/lib/auth.ts:53-72`	`emailAndPassword` and `socialProviders`. No `account` block — see finding 1.
`better-auth/dist/api-CkmycQ2x.mjs:799`	Credential path: `accountId: account.id`, password only.
`better-auth/dist/api-CkmycQ2x.mjs:1034-1060`	Account linking: trust checks, token update, `.filter(v !== undefined)`.
`better-auth/dist/api-CkmycQ2x.mjs:1073-1078`	OAuth insert shape; `accountId: String(userInfo.id)`, `scope: scopes?.join(",")`.
`better-auth/dist/api-CkmycQ2x.mjs:363`	`setTokenUtil` — the `encryptOAuthTokens` gate.
`better-auth/dist/email-otp-CiznqFUN.mjs:541`	Password reset: creates the `credential` account or updates the password.
`better-auth/dist/crypto-*.mjs:132-136`	scrypt `hashPassword` / `verifyPassword` with `constantTimeEqual`.
`src/features/executions/hooks/use-node-status.ts:10`	`Realtime.Subscribe.Token` — the unrelated `refreshToken`.

Method note

Row counts, per-provider column population, the three `accountId` shapes, the multi-provider grouping, all `scope` values, every `accessTokenExpiresAt`, the constraint list from `pg_constraint`, and the column types from `information_schema` were read directly from the project's Neon database. The insert shapes, linking guards, `setTokenUtil` encryption gate, and `accountId` derivation were read from the installed `better-auth` package. The claim that no application code reads the tokens came from an exhaustive search of `src/`, not from inspection. No finding on this page is assumed; where a claim could not be verified from the installed package, it is stated as observed-in-this-database rather than as a specification.

— end of document —